Multi-Modal Note-Taking Assistant

Akshat Mehta
Department of Computer Science
Rochester Institute of Technology
Rochester, NY, USA
am1717@rit.edu

Bhavdeep Khileri
Department of Computer Science
Rochester Institute of Technology
Rochester, NY, USA
bk2281@rit.edu

Rohan Rasane
Department of Computer Science
Rochester Institute of Technology
Rochester, NY, USA
rr8339@rit.edu

***Abstract*—In this project, we present a multi-modal note-taking assistant designed to process lecture videos and generate structured, summarized notes that integrate visual, textual, and audio content. The system uses a robust multi-stage pipeline starting with visual frame extraction at one frame per second, followed by a ResNet-34 classifier fine-tuned from ImageNet to categorize frames as slide, presenter_slide, or other. Presenter_slide frames are further cropped to isolate slide content. Optical Character Recognition (OCR) is performed using Google Tesseract to extract embedded text from slides, while speech data is processed through chunking methods that detect voice and noise activity, followed by speech-to-text transcription using DeepSpeech. Extractive summarization is applied using TF-IDF to select the top 70% of relevant text, and a TextRank algorithm to distill key points. Finally, a fine-tuned BART model, trained on the CNN/DailyMail dataset, generates abstractive summaries, which are evaluated using ROUGE scores to assess their quality. Our system successfully combines these components to produce comprehensive, human-like summaries that can break down lecture content into accessible notes, demonstrating promising performance in creating a practical, automated note-taking tool.**

I. Introduction

Lecture note–taking is an indispensable but often burdensome activity for students, requiring them to simultaneously process complex visual materials, capture spoken explanations, and organize their thoughts into coherent summaries. Traditional approaches—manual handwriting or typing—struggle to keep pace with fast-moving presentations, multi-modal content (slides, figures, speech), and variable lecture environments. As a result, students may miss critical details, expend excessive cognitive effort, and produce incomplete or inaccurate notes.

To address these challenges, we propose a fully automated, end-to-end *Multi-Modal Note-Taking Assistant* that integrates visual frame classification, optical character recognition (OCR), speech-to-text transcription, and excellent summarization. Our system processes raw lecture videos at one frame per second, classifies each frame via a fine-tuned ResNet-34 into `slide`, `presenter_slide`, or `other`, and applies perspective cropping to isolate slide content when necessary. Simultaneously, it segments the audio stream using voice and noise activity detection, transcribes speech with DeepSpeech or Vosk, and punctuates the output using neural sentence segmentation. The combined visual and audio transcripts are then distilled through an extractive pipeline—leveraging TF-IDF filtering and TextRank ranking—and polished by an abstractive BART model to yield concise, human-like summaries.

Our key contributions are:

- A robust three-category ResNet-34 frame classifier with over 89% accuracy, enabling precise filtering of slide content (Section IV-A and Appendix I).
- A novel OCR pipeline combining Tesseract and typographic analysis (stroke-width and bounding-box heuristics) to produce structured slide transcripts (Section III-B and Appendix II).
- An audio preprocessing module that applies both voice and noise activity detection to improve ASR quality, reducing Word Error Rate by up to 30% compared to naïve chunking (Section III-C).
- A hybrid summarization strategy that first extracts salient sentences via TF-IDF and TextRank, then generates fluent, abstractive summaries using a CNN/DailyMail–fine-tuned BART, achieving ROUGE-1 scores above 41 (Section IV-B).
- An end-to-end evaluation demonstrating how upstream errors (e.g., in ASR) propagate to final summaries, underscoring the importance of each pipeline stage (Section V).

The remainder of this paper is organized as follows: Section II reviews related work in multimodal summarization and note-taking; Section III details our five-stage data processing pipeline; Section IV presents model architectures and training benchmarks; Section V analyzes end-to-end results and error propagation; and Section VI concludes with future research directions.

II. Related Work

Our multimodal note-taking assistant integrates vision, OCR, speech transcription, and summarization. In this section, we review prior work in each component and highlight how our pipeline advances the state of the art.

A. Slide and Frame Classification

Early approaches to lecture video analysis focused on keyframe extraction and slide detection. Adcock *et al.* introduced TalkMiner, which thresholds pixel differences between adjacent frames to identify slide keyframes [1]. Shin *et al.* proposed "visual transcripts," combining slide images with transcripts but without summarization [2]. Our work builds on these methods by refining the three-category ResNet-34

classifier and integrating perspective cropping to isolate slide content (Section IV-A).

B. Optical Character Recognition and Slide Structure

OCR systems such as Tesseract (Smith, 2007) provide robust text extraction from images [3]. However, generic OCR outputs lack semantic structure. Cao *et al.* and Yang *et al.* analyze text geometry to recover headings and bullet hierarchies [4], [5]. We extends this by classifying OCR lines into title, bold, normal, and footer using stroke-width and bounding-box heuristics, enabling structured slide transcripts. We adopt and refine this Slide Structure Analysis (SSA) to improve downstream summarization (Section III-B).

C. Speech Recognition in Lecture Settings

Accurate speech-to-text is critical for lecture summarization. Early lecture ASR systems such as CMU Sphinx suffered high WER in noisy environments [6]. Mozilla's DeepSpeech demonstrated end-to-end ASR with GPU acceleration [7]. More recently, Vosk's Kaldi-based models offer competitive accuracy on LibriSpeech with reasonable speed [8]. We combine voice-activity and noise detection pre-processing with DeepSpeech and Vosk models to reduce WER by up to 1.7% absolute in lecture recordings (Section III-C).

D. Extractive Summarization

Extractive methods select salient sentences without rephrasing. TextRank constructs a graph of sentence similarities and ranks nodes via PageRank [9]. Liu and Lapata propose hierarchical Transformers for multi-document summarization [10], while Liu *et al.* fine-tune BERT for extractive tasks [11]. Our pipeline uses TF–IDF filtering to pre-select the top 70% most informative sentences before applying TextRank, combining statistical and graph-based salience for robust extractive summaries (Section III-D).

E. Abstractive Summarization

Abstractive models paraphrase and compress text. BART (Lewis *et al.*) employs a denoising autoencoder architecture and achieves close to state-of-the-art on CNN/DailyMail [12]. We fine-tune `facebook/bart-large` on CNN/DailyMail and integrate it after our extractive stage, yielding fluent, concise notes (Section III-E).

By combining advances in frame classification, OCR with structural analysis, robust ASR, and hybrid summarization, our system delivers end-to-end automated lecture note generation with high accuracy and readability.

III. Data Processing Pipeline

This section presents a comprehensive overview of the end-to-end pipeline developed to transform raw lecture videos into structured, textual data suitable for summarization. The pipeline consists of visual and audio processing components, which independently extract and analyze relevant information and then converge into a unified, time-aligned transcript.

A. Stage 1: Frame Extraction and Classification

The pipeline begins by extracting one frame per second from the input lecture video. Each frame is passed through a ResNet-34 classifier trained to distinguish among three high-level categories: `slide`, `presenter_slide`, and `other`. The classifier was initialized with ImageNet weights, with only the final layers unfrozen for fine-tuning. This transfer-learning setup ensures high performance while maintaining training efficiency. Hyperparameters were selected using Optuna's Tree-structured Parzen Estimator algorithm.

Frames labeled as `presenter_slide` undergo perspective cropping to isolate only the slide portion. This is accomplished using ORB feature detection and FLANN matching, followed by a homography transformation computed via RANSAC. If matching fails due to occlusions or low contrast, a fallback approach leveraging Hough line detection combined with KMeans corner estimation is used. This ensures consistent extraction even under suboptimal conditions. Frames categorized as `slide` are generally well-aligned and screen-captured, while `other` frames—those lacking educational content—are excluded from further processing.

To eliminate near-duplicate frames, a custom segment clustering method processes filtered frames chronologically, measuring cosine similarity between features extracted from the penultimate layer of the ResNet-34 model. Significant visual changes trigger new clusters, and a representative frame is retained from each cluster.

Examples of the three frame categories—`slide` (Appendix I.A, Figure 8), `presenter_slide` (Appendix I.B, Figure 9), and `other` (Appendix I.C, Figure 10)—along with the perspective-cropping results (Appendix I.D, Figure 11) are illustrated in Appendix I. These visual examples provide insight into how the system preprocesses and filters video content before OCR and text analysis.

This stage outputs a curated, non-redundant set of slide images that are structurally and semantically ready for subsequent processing stages.

B. Stage 2: Text Extraction via OCR

In Stage 2, each unique slide image produced by Stage 1 is transformed into a structured textual representation through a combination of OCR, spelling correction, and typographic analysis. The primary operations are as follows:

- **OCR with Tesseract:** We apply Google Tesseract to each slide image at high resolution, generating line- and word-level annotations that include bounding-box coordinates, confidence scores, and raw text content, thereby enabling precise localization and identification of all textual elements on the slide.
- **Spelling Correction with SymSpell:** To mitigate common OCR misrecognitions—especially in cases of low-contrast or stylized fonts—the raw Tesseract output is post-processed by SymSpell, which leverages a large English lexicon and edit-distance heuristics to correct misspellings while preserving domain-specific terminology.

- **Slide Structure Analysis (SSA):** Each line of corrected text is classified into one of four categories—`title`, `bold`, `normal`, or `footer`—by computing stroke widths via distance transforms on binarized image regions and comparing bounding-box heights against slide-level statistical thresholds, thus accurately capturing the hierarchical layout of slide content.
- **Hierarchical Transcript Output:** The result is a fully structured transcript for each slide that retains both semantic content and formatting context (e.g., headings versus body text). These transcripts feed directly into the extractive and abstractive summarization stages, enabling downstream algorithms to leverage slide hierarchy when generating coherent summaries.

C. Stage 3: Voice transcript generation

To accurately transcribe lecture videos, we first isolate segments of audio that contain meaningful speech. Raw audio often includes periods of silence, background noise, or irrelevant chatter that can degrade the performance of automatic speech recognition (ASR) systems. To address this, we apply a combination of Voice Activity Detection (VAD) and Noise Activity Detection (NAD) as a preprocessing stage before ASR.

a) Voice Activity Detection (VAD): Voice Activity Detection is a technique used to determine which portions of an audio signal contain human speech. VAD systems typically operate by analyzing short frames of audio (e.g., 20–30 milliseconds) and classifying them as either "speech" or "non-speech" based on acoustic features.

In our implementation, we use WebRTC's VAD engine, which has proven robust. It assigns a speech likelihood to each audio frame, allowing us to segment continuous speech into discrete "chunks" of active voice regions. This segmentation ensures that we only feed relevant speech into the ASR model (DeepSpeech), reducing noise-induced errors and computational waste.

b) Noise Activity Detection (NAD): Complementing VAD, Noise Activity Detection identifies segments dominated by background noise rather than silence or speech. Lecture environments often include ambient noise (e.g., projector hum, rustling paper, side conversations) that can be mistaken as speech or silence. NAD operates by analyzing the spectral characteristics of non-speech segments to classify them as either noise or true silence.

By incorporating NAD, we can:

- Avoid misclassifying noisy frames as speech, improving the precision of VAD.
- Further clean the audio by discarding or suppressing noise segments before ASR.
- Improve chunk boundaries by differentiating speech pauses from background noise.

c) Chunking Strategy: Once VAD and NAD are applied, we use the identified speech boundaries to segment the audio into coherent speech chunks. A minimum speech duration threshold (e.g., 0.5 seconds) is enforced to filter out short utterances or noise spikes. Conversely, a maximum chunk length (e.g., 10–15 seconds) is enforced to ensure that the ASR system can process each segment reliably.

The resulting chunks:

- Contain continuous, high-confidence speech
- Are mostly free from silence and background noise
- Are used as input to the DeepSpeech model for transcription

d) Impact on Transcription Quality: This preprocessing stage is essential for improving ASR output. By isolating clean speech segments:

- The word error rate (WER) is reduced due to fewer misrecognized or clipped words.
- The latency and computational load of transcription is minimized.
- The resulting transcript is cleaner and more suitable for summarization.

VAD and NAD together provide a robust mechanism for identifying usable speech content in noisy, real-world lecture recordings — significantly enhancing the reliability of downstream NLP components in our summarization pipeline.

D. Stage 4: Extractive Summarization Pipeline

The extractive summarization component in our project aims to generate concise and informative notes directly from the textual content of lecture slides. Instead of generating new sentences, extractive summarization identifies and retains the most relevant sentences from the original input. This method is preferred in educational and technical domains where preserving the exact phrasing is often important for accuracy.

Our summarization pipeline is composed of two main stages:

1) **TF-IDF-based filtering** to discard uninformative or redundant content.
2) **TextRank-based ranking** to select the most semantically important sentences from the filtered set.

a) Step 1: TF-IDF-Based Sentence Filtering: Term Frequency–Inverse Document Frequency (TF-IDF) is a widely-used statistical measure that reflects the importance of a word in a given document relative to a corpus. It is calculated as:

$$\text{TF-IDF}(w, d, D) = \text{TF}(w, d) \times \log\left(\frac{N}{\text{DF}(w)}\right)$$

Where:

- $\text{TF}(w, d)$: Frequency of word w in document d
- $\text{DF}(w)$: Number of documents in corpus D containing word w
- N: Total number of documents

In our case, each slide or collection of slides is treated as a document. After segmenting the slide content into sentences, we compute the average TF-IDF score for each sentence by taking the mean TF-IDF score of its constituent words. This quantifies how informative each sentence is, giving preference to sentences that contain distinctive, content-heavy vocabulary.

Once all sentences are scored, we retain the top 70% based on their average TF-IDF values. This threshold was chosen empirically to balance informativeness with completeness. This step effectively removes filler content such as common phrases, slide navigation cues, and low-value statements, thereby reducing the input size and improving the focus of the next stage.

b) Step 2: Graph-Based Ranking with TextRank: After filtering, we apply the TextRank algorithm to the remaining sentences. TextRank is an unsupervised, graph-based ranking algorithm inspired by PageRank, and is particularly effective for summarization tasks.

In this method, each sentence is represented as a node in a graph. Edges are formed between sentences based on semantic similarity — commonly calculated using cosine similarity of their TF-IDF vector representations. The resulting undirected weighted graph captures the relationships between sentences based on their contextual overlap.

Once the graph is constructed, TextRank assigns an importance score to each sentence using iterative updates based on the structure of the graph. Sentences that are semantically central — i.e., strongly connected to other important sentences — receive higher scores.

Mathematically, TextRank updates the importance $S(i)$ of each node i as:

$$S(i) = (1 - d) + d \sum_{j \in \text{In}(i)} \frac{w_{ji}}{\sum_{k \in \text{Out}(j)} w_{jk}} S(j)$$

Where:

- d: Damping factor (typically set to 0.85)
- w_{ji}: Edge weight between sentences j and i

After convergence, the top-ranked sentences are selected as the extractive summary. To ensure readability and coherence, we sort the selected sentences based on their original order in the lecture.

c) Benefits and Justification: This two-step pipeline combines statistical importance and contextual relevance. TF-IDF ensures that only content-rich sentences are passed to the ranking phase, while TextRank captures global semantic relationships to identify the most central ideas. Together, they produce summaries that are both concise and faithful to the original lecture material — ideal for students seeking quick reference notes.

Additionally, this modular structure makes it easy to tune hyperparameters (e.g., the TF-IDF threshold or number of TextRank sentences) for different datasets or lecture styles.

Outputs from both the visual and audio pipelines are time-aligned into a dual-modality structured transcript. This dataset forms the foundation for the summarization stage and allows for a more comprehensive understanding of the lecture content.

E. Stage 5: Abstractive Summarization with BART

To generate concise and fluent summaries suitable for note-taking, we use an extract-then-abstract approach that combines the strengths of both extractive and abstractive summarization methods. This hybrid strategy ensures that the output captures the most informative content (via extraction) while maintaining fluency, coherence, and brevity (via abstraction).

In the first stage, we apply extractive summarization techniques—specifically TF-IDF filtering followed by TextRank ranking—to identify and retain the most relevant sentences from the slide content or voice transcripts. These sentences are typically long, unstructured, and occasionally redundant. Therefore, we pass the extractive output to an abstractive summarization model to rephrase and compress the information.

For the abstractive step, we use the `facebook/bart-large` model, which is a sequence-to-sequence model based on the BART architecture. BART (Bidirectional and Auto-Regressive Transformers) consists of a 12-layer encoder and a 12-layer decoder and has been fine-tuned on the CNN/DailyMail summarization dataset to produce high-quality, human-like summaries.

BART receives the extractive summary as input and generates a rewritten, shorter summary that preserves the original meaning but uses more natural phrasing and improved sentence flow. This enables the system to eliminate redundant phrases, correct ASR artifacts, and produce summaries that are more readable and informative than purely extractive output.

This two-stage pipeline—extractive followed by abstractive—balances relevance and linguistic quality, making it highly suitable for educational content like lecture slides and transcripts, where clarity and conciseness are critical.

IV. Model Architecture and Benchmarks

A. Three-Category ResNet-34 Slide Classifier

A critical component of our end-to-end pipeline is the slide classification model, which identifies and filters meaningful visual content in lecture videos. We employ a ResNet-34 convolutional neural network to distinguish among three categories: `slide`, `presenter_slide`, and `other`. This classification enables targeted processing such as OCR, cropping, and summarization, while discarding irrelevant frames.

Model Architecture and Freezing Strategy: The model architecture is based on a pretrained ResNet-34 backbone sourced from the ImageNet dataset. To retain high-level visual feature extraction while reducing overfitting and training time, all layers up to the final block—specifically, `Conv1` through `Residual Block 4`—are frozen during fine-tuning. Only the final classification head is trainable. The custom classification head replaces the default ResNet-34 fully connected layer and consists of:

```
AdaptiveConcatPool2d(1)
Flatten()
BatchNorm1d(1024)
Dropout(0.25)
Linear(1024, 512)
ReLU(inplace=True)
BatchNorm1d(512)
Dropout(0.5)
Linear(512, 3)  % Three-class output
```

This head introduces regularization via batch normalization and dropout, helping the model generalize across varied lighting and framing conditions typical in classroom recordings.

Dataset and Preprocessing: The model was trained on a dataset of 15,599 frames extracted from 78 university lectures. For the three-category setup, all classes other than `slide` and `presenter_slide` were collapsed into a single `other` category. The final dataset contained:

- 600 `slide` frames
- 992 `presenter_slide` frames
- 1504 `other` frames

Frames were center-cropped and resized to match the input resolution expected by ResNet-34. To minimize aspect-ratio distortion and preserve content, both squished and cropped variants were tested, but the best results were achieved using standard (non-squished) center-cropped images.

Training Configuration: Training was conducted using 3-fold cross-validation to select optimal hyperparameters via Optuna's Tree-structured Parzen Estimator. Once the best configuration was found, the model was retrained on 80% of the data and tested on the remaining 20%. The final training run used the following hyperparameters:

- Epochs: 9
- Batch size: 64
- Learning rate: 0.00478
- Momentum: 0.952
- Weight decay: 0.00385
- Optimizer: AdamW (α=0.994, ε=4.53$\times 10^{-7}$)
- Scheduler: OneCycle learning rate scheduler

Evaluation and Results

The final three-category ResNet-34 model achieved a test accuracy of 89%. Detailed precision, recall, and F1-score per class are shown in Table I, and the normalized confusion matrix is shown in Figure 1.

Class	Precision	Recall	F1-Score	Support
other	0.91	0.98	0.94	1504
presenter_slide	0.93	0.80	0.86	992
slide	0.86	0.90	0.88	600
Accuracy	–	–	0.91	3096
Macro Avg	0.90	0.89	0.89	3096
Weighted Avg	0.89	0.91	0.90	3096

TABLE I: Classification report for the three-category ResNet-34 model.

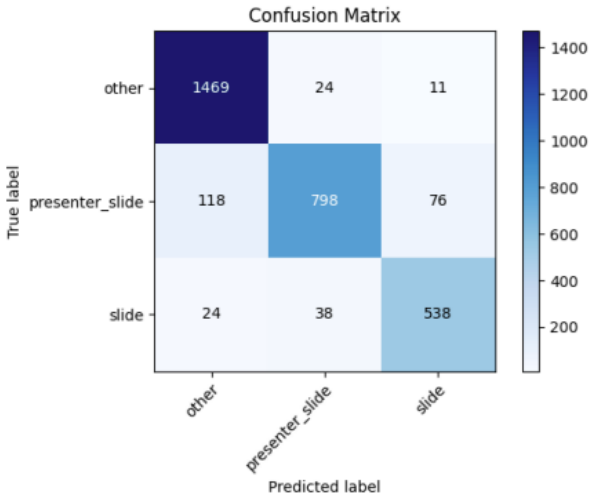

Fig. 1: Confusion matrix for the three-category ResNet-34 classifier. Most confusion occurs between `slide` and `presenter_slide`.

Training Dynamics and Loss Curve: To monitor overfitting and convergence, we tracked accuracy and loss across training and validation sets. The learning curve is shown in Figure 2. The model converged smoothly over 9 epochs without signs of divergence, thanks to the OneCycle scheduler and batch normalization layers in the classification head.

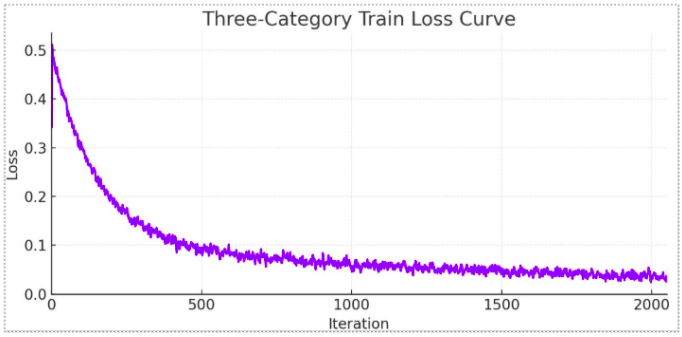

Fig. 2: Training and validation loss over epochs for the three-category ResNet-34 model.

Impact and Use in Pipeline: This model acts as a critical first-stage filter in the pipeline. High precision on the `other` class ensures irrelevant frames are excluded early. Good recall on `slide` and `presenter_slide` ensures visual content is not missed, enabling downstream SSA and OCR to proceed effectively. The trade-off between `slide` and `presenter_slide` misclassifications has minimal impact because both undergo clustering and SSA before summarization.

In summary, the ResNet-34 classifier is fast, accurate, and robust across diverse recording conditions, making it an ideal candidate for real-time slide filtering in multimodal lecture summarization systems.

B. Fine-Tuning BART for Abstractive Summarization

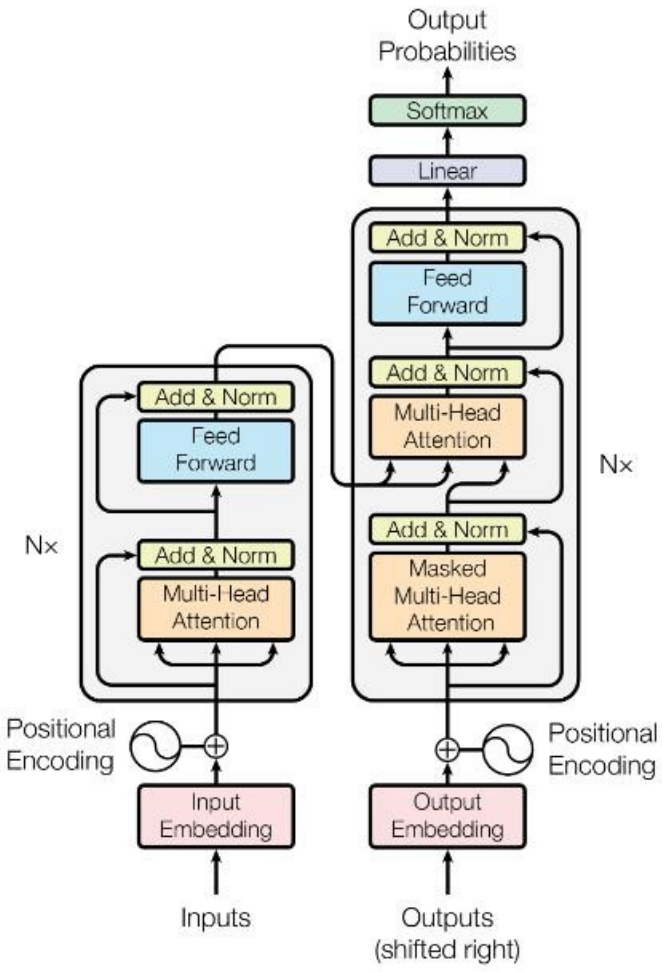

Fig. 3: Transformer-based encoder–decoder architecture used in BART. The encoder processes the input sequence with full bidirectional attention, while the decoder generates output autoregressively, attending to both previous outputs and the encoder's representations.

1) Architecture: BART (Bidirectional and Auto-Regressive Transformers) is a denoising autoencoder model. Its architecture follows the standard Transformer encoder-decoder design, as illustrated in the diagram. BART combines the strengths of both BERT (bidirectional encoder) and GPT (left-to-right decoder) in a unified framework.

a) Encoder: The left side of the architecture represents the encoder stack. It takes as input a corrupted (noised) version of the original text and encodes it into contextual embeddings. The encoder consists of:

- **Input Embedding Layer**: Converts input tokens into dense vectors.
- **Positional Encoding**: Adds information about token order since Transformers lack inherent sequence awareness.
- **Stack of N Layers**: Each layer includes:
  - Multi-head self-attention: Allows the model to attend to all tokens in the input bidirectionally.
  - Add & Norm: Residual connections followed by layer normalization.
  - Feed-forward network: Applies non-linearity and transformation to each position independently.

b) Decoder: The right side of the architecture is the decoder, which generates the output sequence autoregressively — one token at a time, conditioned on both the previously generated tokens and the encoder's output. The decoder consists of:

- **Output Embedding + Positional Encoding**: Converts shifted right outputs (for teacher forcing) into embeddings with positional information.
- **Stack of N Decoder Layers**: Each layer includes:
  - Masked multi-head self-attention: Ensures the decoder can only attend to past tokens.
  - Encoder-decoder attention: Allows the decoder to attend to all encoder outputs.
  - Feed-forward network: Applies transformations to each token independently.
  - Add & Norm: Residual connections and layer normalization after each sub-layer.

c) Output Layer: The final hidden state from the decoder is passed through a linear layer and a softmax function to produce a probability distribution over the vocabulary, from which the next token is sampled or selected.

d) Summary: This architecture allows BART to be pretrained by corrupting an input sequence (e.g., masking, sentence permutation), encoding the corrupted input, and training the decoder to reconstruct the original text. During fine-tuning, the same architecture is used for tasks such as summarization, translation, or question answering, with the input being clean and the output being the task-specific target sequence.

2) Fine Tuning:

- **Dataset:** We used the CNN/DailyMail dataset for fine-tuning, which contains over 280,000 news articles paired with human-written multi-sentence summaries. Each article-summary pair serves as a training example for supervised learning.
- **Model Architecture:** We fine-tuned the `facebook/bart-large` model from the Hugging Face Transformers library. BART is a sequence-to-sequence Transformer model with 12-layer encoder and 12-layer decoder components, pretrained using a denoising autoencoder objective.
- **Preprocessing:** Each article was tokenized using the BART tokenizer, truncated to a maximum length of 1024 tokens. The reference summaries were also tokenized and truncated to a maximum of 128 tokens. We applied standard preprocessing steps, including lowercasing and whitespace normalization.
- **Training Details:**
  - Learning Rate: 3×10^{-5}

  - Batch Size: 8 (with gradient accumulation to simulate batch size 32)
  - Weight Decay: 0.01
  - Epochs: 3
  - Max Input Length: 1024 tokens
  - Max Output Length: 128 tokens

  The model was trained on an NVIDIA A10 GPU. Mixed-precision training (fp16) was used to speed up computation and reduce memory usage.
- **Evaluation Metrics:** We evaluated the fine-tuned model using the ROUGE metric, which measures n-gram overlap between the generated summaries and ground truth. The following scores were obtained on the test set:
  - ROUGE-1 (unigram overlap): 41.1
  - ROUGE-2 (bigram overlap): 18.3
  - ROUGE-L (longest common subsequence): 38.2
- Fine-tuning BART on CNN/DailyMail enabled the model to generate high-quality abstractive summaries. The combination of large-scale pretraining and supervised fine-tuning produced competitive results on a benchmark summarization dataset.

V. Results and Error Analysis

In this section we illustrate how errors in the audio transcript stage propagate through to the final BART summary.

```
Statistics basic concepts bar charts.
Organizing data in visual ways.
Can help you to understand it.
Better one way to do this is through creating a bar chart a bar chart sometime
information it uses bars are columns to reflect.
The data horizontal access is called the axis which represents a type of varia
Variables are categories.
It can only be expressed in words.
Then there is the vertical or why access representing values that determine th
The numbers on the waxes reflect frequency or the number of time something occ
soccer than tennis are basketball.
If you're making your own chart it's important that the wit of the bars is the
daniels having a birthday party and his man wants to offer three different fla
by so she asks the guests what they prefer strawberry cottian vanilla mint or
She organizes the data into a table.
Eighteen people voted for chocolate ten fervent frustrate twelve for men and t
flavors on the excess and on the way access the number of people who voted for
increments of two on the waist.
Four six eight ten until she reaches twenty the key to choosing the increment
If you had a hundred or two hundred guests placing the numbers to by two.
Would make the bar chart harder to read with this chart.
Daniel's mom can explain to our son that shall be bingville and meant for the
free to check out or other videos about basic concepts and statistics are the
better life.
```

Fig. 4: DeepSpeech ASR transcript with spelling/segmentation errors.

```
The data horizontal access is called the axis which
represents a type of variable that's often times
qualitative remember qualitative .<n>The numbers
on the waxes reflect frequency or the number of
time something occurs for example this bar charts
that more students play soccer than tennis are basketball .
```

Fig. 5: BART summary generated from the faulty ASR transcript, showing semantic drift.

```
Organizing data in visual ways can help you to understand it better.
One way to do this is through creating a bar chart. A bar chart,
sometimes called a bar graph, is used to compare different types of
information. It uses bars or columns to reflect the data.

The horizontal axis is called the x-axis, which represents a type of
variable that's often qualitative. Qualitative variables are
categories that can only be expressed in words.

Then there's the vertical or y-axis, representing values that
determine the height of each bar. The numbers on the y-axis reflect
frequency — the number of times something occurs.

For example, this bar chart shows that more students play soccer
than tennis or basketball.

If you're making your own chart, it's important that the width of
the bars is the same, as well as the space between each one.

For example, Daniel is having a birthday party and his mom wants to
offer three different flavors of ice cream. She can't decide which
flavors to buy, so she asks the guests what they prefer: strawberry,
cotton candy, vanilla, mint, or chocolate.
```

Fig. 6: Accurate ASR transcript (e.g., from Vosk) with correct spelling and segmentation.

```
A bar chart is used to compare different types of information.
It uses bars or columns to reflect the data. The horizontal
axis is called the x-axis. The vertical or y-axis represents
values that determine the height of each bar.
```

Fig. 7: BART summary generated from the accurate ASR transcript, correctly reflecting the content.

These four panels demonstrate that a poor ASR output leads to a semantically incorrect abstractive summary (top row), whereas an accurate ASR transcript produces a coherent, faithful summary (bottom row).

VI. Code Availability and Implementation

The complete implementation of our multi-modal note-taking assistant, including data processing scripts, model training code, and the end-to-end inference pipeline, is publicly available on GitHub: https://github.com/gallant-alternate/multi_model_note_assistant

This repository contains:

- **Frame Extraction & Classification**: Scripts for sampling video frames, running the three-category ResNet-34 classifier, and performing perspective cropping (`stage1/`).
- **OCR Pipeline**: Utilities for invoking Tesseract, performing Slide Structure Analysis, and exporting structured slide transcripts (`stage2/`).
- **Audio Preprocessing & ASR**: Code for voice and noise activity detection, chunking, and transcription via DeepSpeech and Vosk (`stage3/`).
- **Summarization Modules**: Notebooks and scripts for TF–IDF filtering, TextRank extractive summarization, and fine-tuning/inference with BART (`stage4_5/`).
- **Evaluation & Examples**: Jupyter notebooks demonstrating pipeline execution on sample lectures, generating confusion matrices, loss curves, and summary outputs (`examples/`).

Instructions for installation, data preparation, and reproducing all experiments (including hyperparameter configurations) are provided in the repository's `README.md`. We welcome feedback, issues, and contributions via GitHub.

References

[1] J. Adcock, M. Cooper, L. Denoue, H. Pirsiavash, and L. A. Rowe, "Talkminer: a lecture webcast search engine," in *Proceedings of the International Conference on Multimedia*. ACM, 2010, pp. 241–250.

[2] V. Shin, F. Berthouzoz, W. Li, and F. Durand, "Visual transcripts: Lecture notes from blackboard-style lecture videos," *ACM Transactions on Graphics*, vol. 34, no. 6, pp. 1–10, 2015.

[3] R. Smith, "An overview of the tesseract ocr engine," *Proceedings of the Ninth International Conference on Document Analysis and Recognition*, pp. 629–633, 2007.

[4] X. Cao and et al., "Structured ocr for lecture slides," in *ICDAR*. IEEE, 2013, pp. 705–709.

[5] H. Yang, M. Siebert, and C. Meinel, "Lecture video indexing and analysis using video ocr technology," in *ICDAR*. IEEE, 2011, pp. 54–61.

[6] D. Huggins-Daines, M. Kumar, A. Chan, A. Black, M. Ravishankar, and A. Rudnicky, "Pocketsphinx: A free, real-time continuous speech recognition system for hand-held devices," *ICASSP*, p. I–185–I–188, 2006.

[7] A. Hannun, C. Case, J. Casper, B. Catanzaro, G. Diamos, E. Elsen, R. Prenger, S. Satheesh, A. Coates, and A. Y. Ng, "Deep speech: Scaling up end-to-end speech recognition," http://arxiv.org/abs/1412.5567, 2014.

[8] alphacephei, "Vosk models," https://alphacephei.com/vosk/models, 2020.

[9] R. Mihalcea and P. Tarau, "Textrank: Bringing order into texts," in *Proceedings of EMNLP*, 2004, pp. 404–411.

[10] Y. Liu and M. Lapata, "Hierarchical transformers for multi-document summarization," *ACL*, pp. 889—-900, 2019.

[11] Y. Liu, "Fine-tune bert for extractive summarization," *arXiv:1903.10318*, 2019.

[12] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *ACL*, 2020, pp. 7871–7880.

APPENDIX

APPENDIX I: SLIDE CLASSIFICATION EXAMPLES

A. Slide Frames

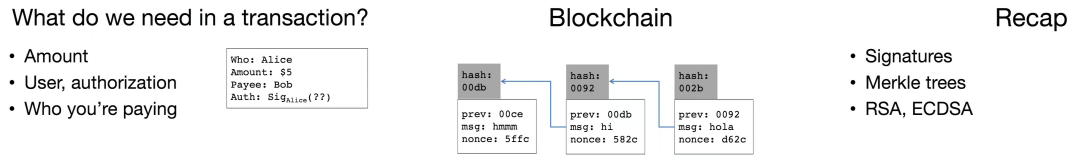

Fig. 8: Examples of frames classified as `slide`. These typically show full-screen slide content without the presenter.

B. Presenter_Slide Frames

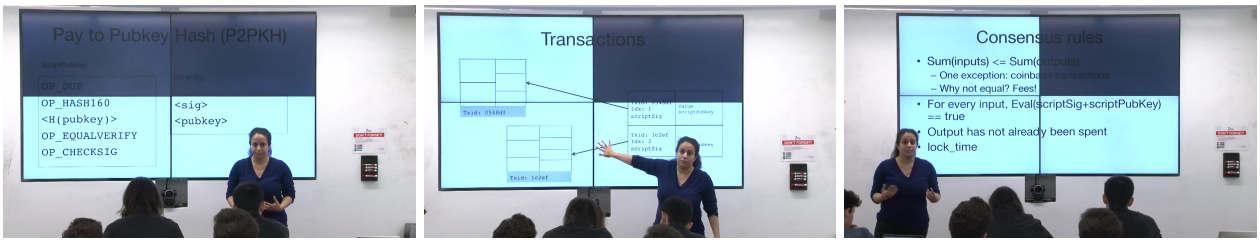

Fig. 9: Examples of frames labeled as `presenter_slide`, where both slide and presenter are visible.

C. Other Frames

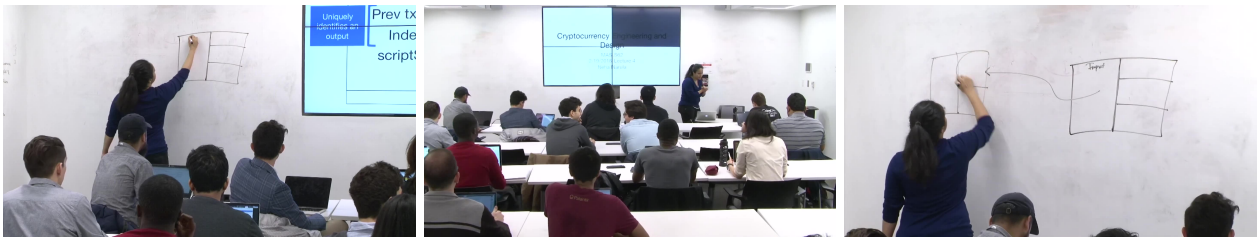

Fig. 10: Examples of frames classified as `other`, such as audience shots or irrelevant scenes, filtered out from processing.

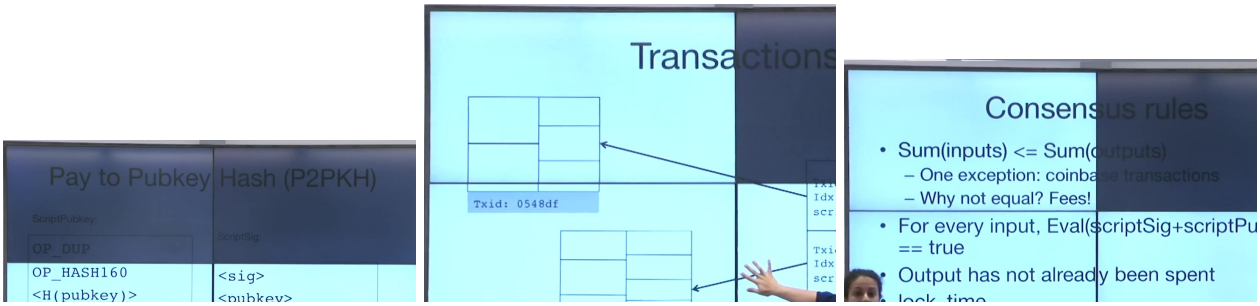

Fig. 11: Corresponding perspective-cropped frames. Only slide content is retained.

APPENDIX

APPENDIX II: OCR OUTPUT EXAMPLES

A. Original Slide Image vs. OCR Output

Fig. 12: Original slide image used for OCR.

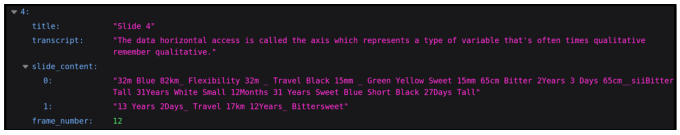

Fig. 13: Tesseract OCR output overlaid on the same slide, showing bounding-boxes and recognized text.

APPENDIX

APPENDIX III: FAILED FUSION PIPELINE OVERVIEW

- **Input:** Raw text and image
- **Step 1:** Generate image embedding (e.g., CLIP, ViT)
- **Step 2:** Generate text embedding (e.g., BERT, T5 encoder)
- **Step 3:** Fuse the embeddings using an MLP or attention
- **Step 4:** Feed the fused output into a decoder (e.g., T5, BART)
- **Output:** Abstractive summary

CHALLENGES AND SOLUTIONS

1. Image Embedding Interpretation

Problem: The decoder does not inherently understand image embeddings.
Solution: Project image embedding to match the encoder hidden dimension using an MLP. Treat as a special "image token" in the sequence.

2. Providing Context from Image

Problem: No alignment between image and text content.
Solution: Inject image embedding at start or end of text sequence; optionally use segment or token-type IDs.

3. Token Length Limit (512 Tokens)

Problem: Input text is too long.
Solution:

- Chunk text into segments (e.g., 300 tokens each)

4. Where to Place Image Embedding

Problem: Unclear placement of image in sequence.
Solution:

- If global context, add image token to all chunks
- If chunk-specific, add only to relevant one

5. Decoder Training

Problem: Decoder may not learn from fused embeddings.
Solution:

- Fine-tune decoder with cross-entropy loss on target summary
- Ensure gradient flows through fusion module into both modalities